Лекция 5. Инъекции в веб-приложениях

Продолжительность: 60 минут. Цель — научиться находить точки, где пользовательский ввод меняет смысл SQL-, PHP- или XML-выражения, и воспроизводить атаки на локальном учебном стенде.

ВЕБ-БЕЗОПАСНОСТЬ | CTF / ПЕНТЕСТ | ЛЕКЦИЯ 5



План лекции

01

Принцип инъекции

Как пользовательский ввод меняет смысл запроса или команды

02

SQL-инъекции

Краткий SQL, механизм уязвимости, обход аутентификации

03

PHP-инъекции

eval(), system(), command injection, reverse shell

04

XML-инъекции

XXE и XPath-инъекция

05

Другие типы

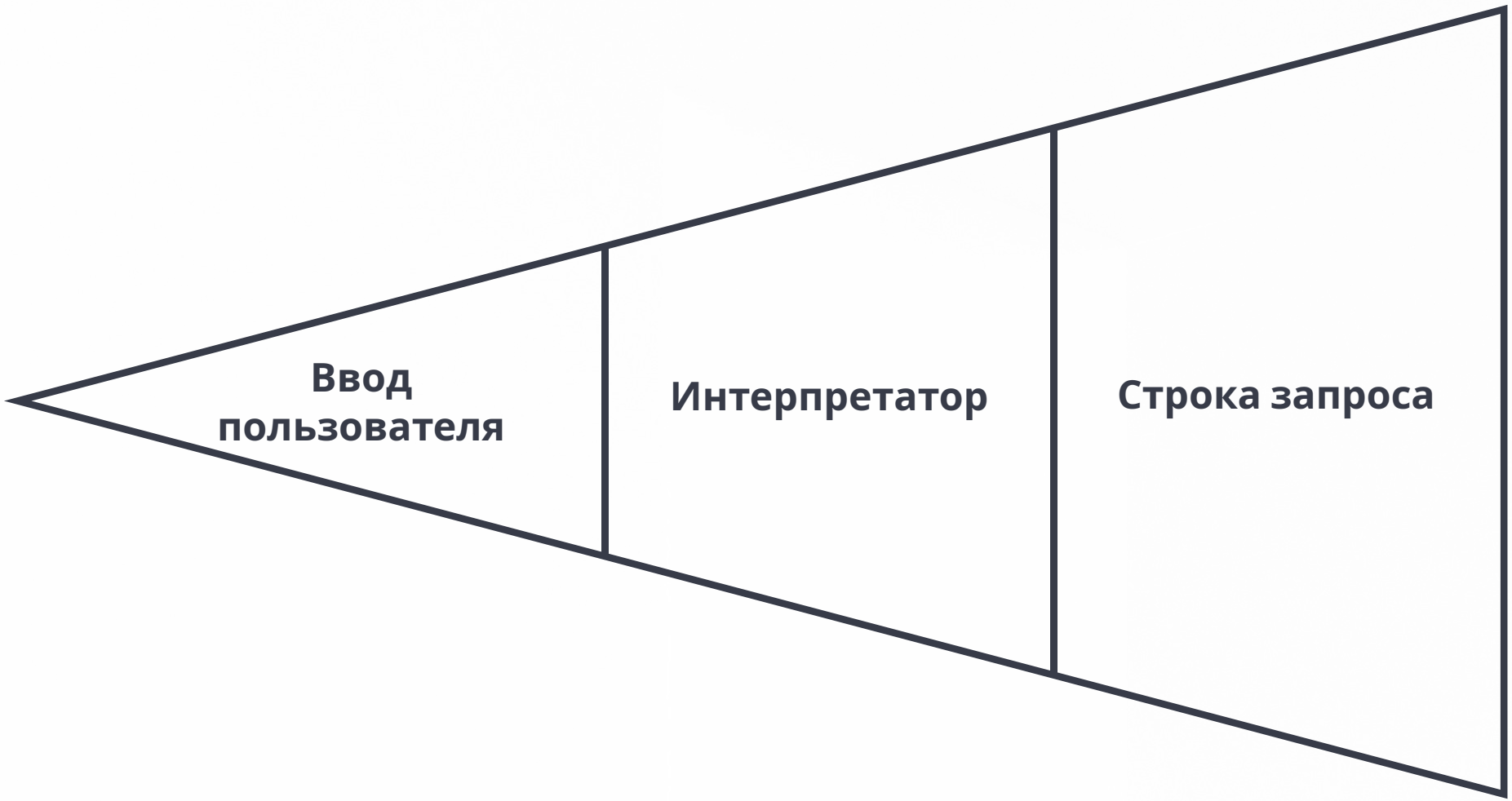
LDAP, NoSQL, SSTI, CRLF и общий алгоритм анализа

РАЗДЕЛ 1

Принцип инъекции

Понять базовый механизм — значит понять любой тип инъекции. Всё начинается с одной точки: ввод пользователя попадает в строку, которую разбирает другой компонент системы.

Как возникает инъекция



Ключевой момент: атакующий **не угадывает пароль** — он подбирает такой ввод, который закрывает начатую программой конструкцию и добавляет свою логику.

Источники пользовательского ввода

Любой из этих каналов может стать точкой инъекции, если данные вставляются в строку запроса без санитизации.



Поля форм

Логин, пароль, поисковые строки, текстовые области — самые распространённые точки входа.



URL-параметры

Параметры GET-запроса в адресной строке: `?id=1&category=web`



Cookie и заголовки

HTTP-заголовки и значения cookie, которые приложение читает и вставляет в запросы.



JSON и файлы

Тела API-запросов, загружаемые XML-документы, данные из конфигурационных файлов.

Что искать при анализе CTF-задачи

Точки ввода

Поля формы, URL-параметры, заголовки, cookie и JSON-поля — всё, что приложение принимает от пользователя.

Место конкатенации

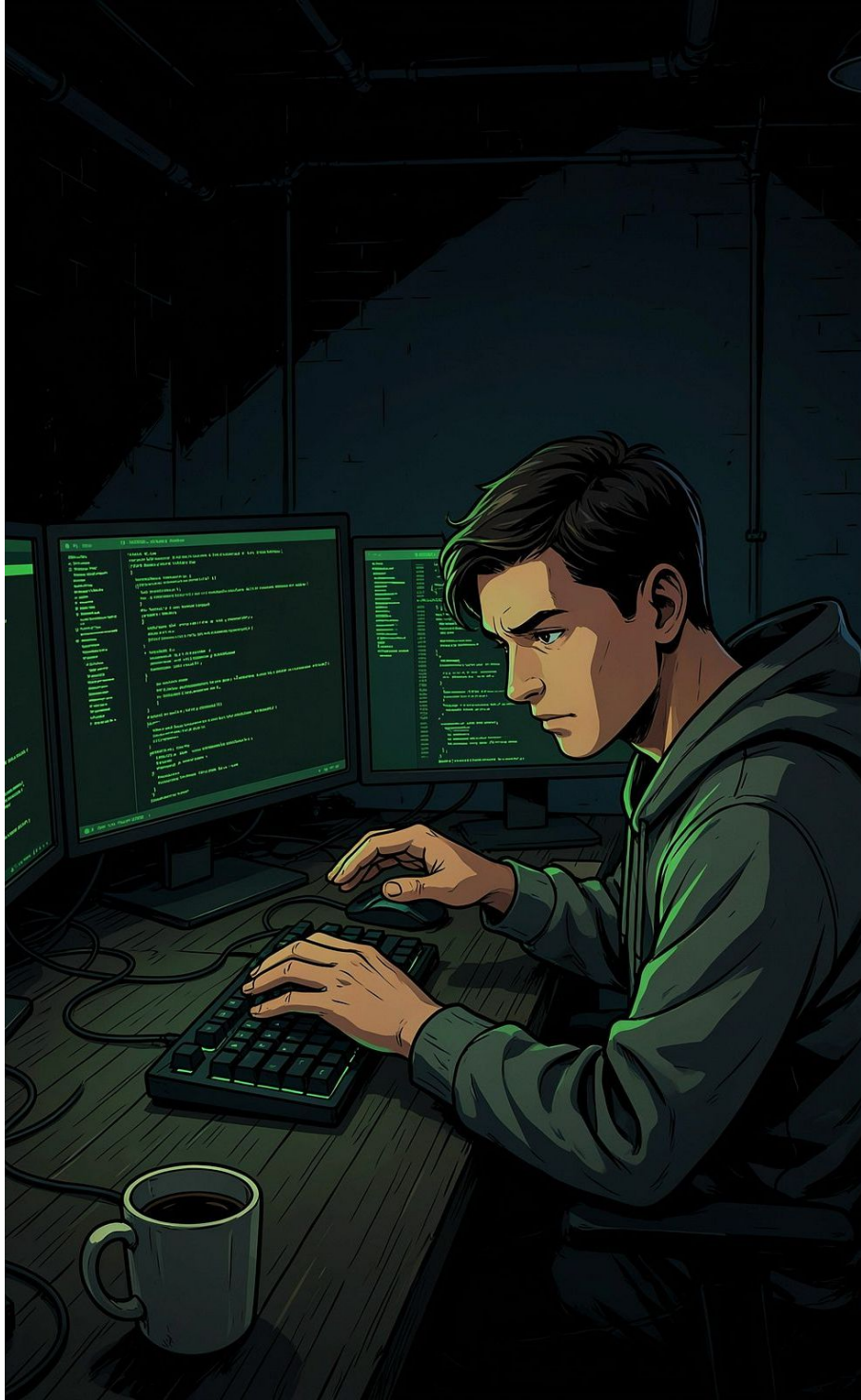
Участок кода, где пользовательские данные склеиваются со строкой запроса или команды.

Интерпретатор

Язык или компонент, который разберёт итоговую строку: SQL, PHP, XPath, LDAP, Bash.

Синтаксис

Кавычки, скобки, комментарии, тип данных — всё, что позволяет «сломать» исходную конструкцию.



РАЗДЕЛ 2

SQL и SQL-инъекции

SQL — наиболее часто встречающийся вектор инъекций в веб-приложениях. Разберём синтаксис, затем механизм уязвимости и практику обхода входа.

Краткий справочник SQL

Понимание базовых конструкций SQL необходимо для грамотного составления инъекционных строк.

Конструкция	Назначение
`SELECT`	Получить строки и столбцы из таблицы
`FROM`	Указать таблицу-источник данных
`WHERE`	Отфильтровать строки по условию
`ORDER BY`	Отсортировать результат по столбцу
`UNION SELECT`	Объединить результаты двух совместимых запросов
`--`	Начать однострочный комментарий (PostgreSQL и другие СУБД)

Пример обычного SQL-запроса

Запрос к базе задач

```
SELECT id, title, points
FROM tasks
WHERE category = 'web'
ORDER BY points DESC;
```

Что происходит

Сервер обращается к таблице `tasks`, фильтрует строки по категории `web` и возвращает результат, отсортированный по убыванию очков. Все данные — статичные, контролируемые разработчиком.

Инъекция появляется, когда значение `'web'` заменяется пользовательским вводом без обработки.

Как возникает SQL-инъекция

В SQLInjectionLab сервер формирует запрос входа по шаблону, подставляя значения полей напрямую:

```
SELECT id, login, display_name, private_note
FROM users
WHERE login = '<логин>'
  AND password = '<пароль>'
```

⚠ Поля формы становятся частями SQL-выражения. Одна кавычка в пользовательском вводе способна сломать синтаксис и изменить логику всего запроса.

Практика: обход входа в SQLInjectionLab

1 Запустить стенд

Запустите стенд из `Les-5/SQLInjectionLab` и откройте `http://127.0.0.1:8081/login` в браузере.

2 Ввести логин

В поле **Логин** введите: `admin`

3 Ввести инъекцию

В поле **Пароль** вставьте: `' OR '1'='1' --`

4 Получить флаг

В личном кабинете откроется закрытая заметка с флагом CTF.

Разбор инъекционного запроса

Посмотрим, что попадает в базу данных после подстановки:

```
SELECT id, login, display_name, private_note
FROM users
WHERE login = 'admin'
  AND password = '' OR '1'='1' -- '
```

Закрытие строки

Первая `'` закрывает значение пароля, вырываясь из строки.

Добавление условия

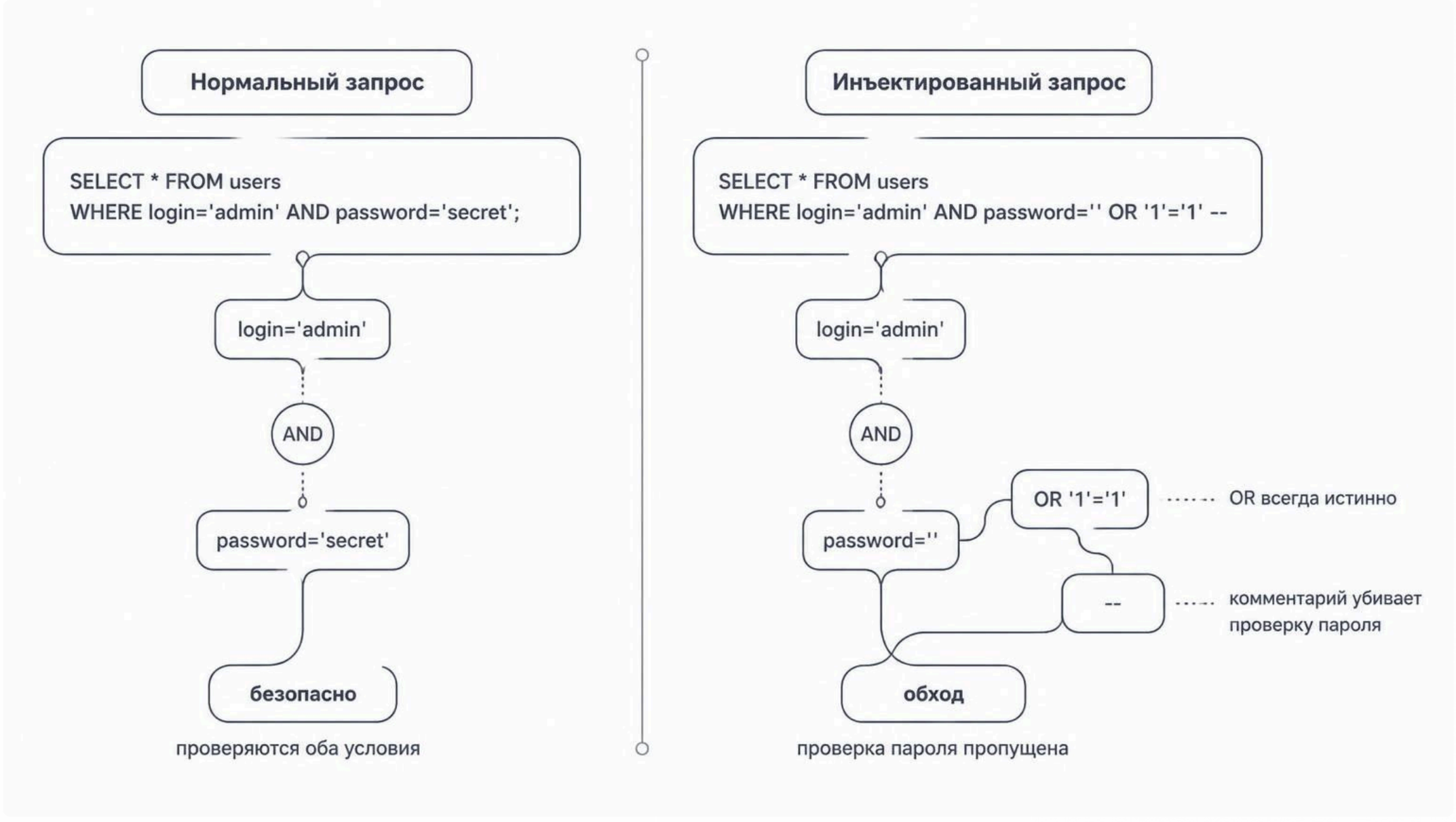
`OR '1'='1'` всегда истинно — условие WHERE выполняется для любой записи.

Комментарий

`--` превращает остаток исходного запроса в комментарий, игнорируя закрывающую кавычку.

В PostgreSQL после `--` обязателен пробел — он оставлен в строке ввода выше.

Визуализация: как инъекция меняет запрос



РАЗДЕЛ 3

PHP-инъекции

PHP-инъекция — ситуация, когда пользовательский текст попадает в исполняемый PHP-код или в команду операционной системы, запускаемую из PHP-скрипта.

Опасные функции PHP

eval()

Выполняет строку как PHP-код. Самая прямая точка инъекции — пользовательский ввод в аргументе.

include / require

Подключают файлы. При уязвимости позволяют загрузить и выполнить произвольный PHP-файл (LFI/RFI).

system() / exec()

Передают строку оболочке ОС и возвращают вывод — прямой путь к Command Injection.

shell_exec()

Аналог `system()`, возвращает весь вывод команды строкой. Часто встречается в «диагностических» страницах.



Учебный калькулятор с eval()

Исходный код страницы

```php
<?php
$expression = $_POST['expression'] ?? '0';
eval('$result = ' . $expression . ';');
echo $result;
```

Логика уязвимости

Скрипт ожидает арифметическое выражение вроде `2+2` и выполняет `$result = 2+2;`. Однако пользователь может подать любой PHP-код.

Переменная `$expression` вставляется в строку без какой-либо проверки или экранирования.

Эксплуатация eval(): пошагово

1

2

3

Ввод атакующего

`1; echo "PHP code executed"; //`

Собранная строка

`$result = 1; echo "PHP code executed"; //;`

Результат

В ответе появляется: `PHP code executed` — подтверждение выполнения произвольного кода.

В CTF появление ожидаемой строки в ответе подтверждает уязвимость. Дальше исследуют только содержимое специально подготовленного контейнера.

Command Injection через system()

Диагностическая страница может содержать следующий код:

```
<?php system('ping -c 1 ' . $_GET['host']);
```

Легитимный запрос

При `host=127.0.0.1` выполняется:

```
ping -c 1 127.0.0.1
```

Стандартный пинг — всё работает корректно.

Инъекционный запрос

При host=127.0.0.1; id выполняется:

```
ping -c 1 127.0.0.1; id
```

В ответе появится идентификатор пользователя веб-процесса.

⊗ Никогда не применяйте этот пример вне собственного изолированного контейнера.

Разделители команд в Bash

Точка с запятой `;`

Выполняет вторую команду всегда, независимо от результата первой.

Двойной амперсанд `&&`

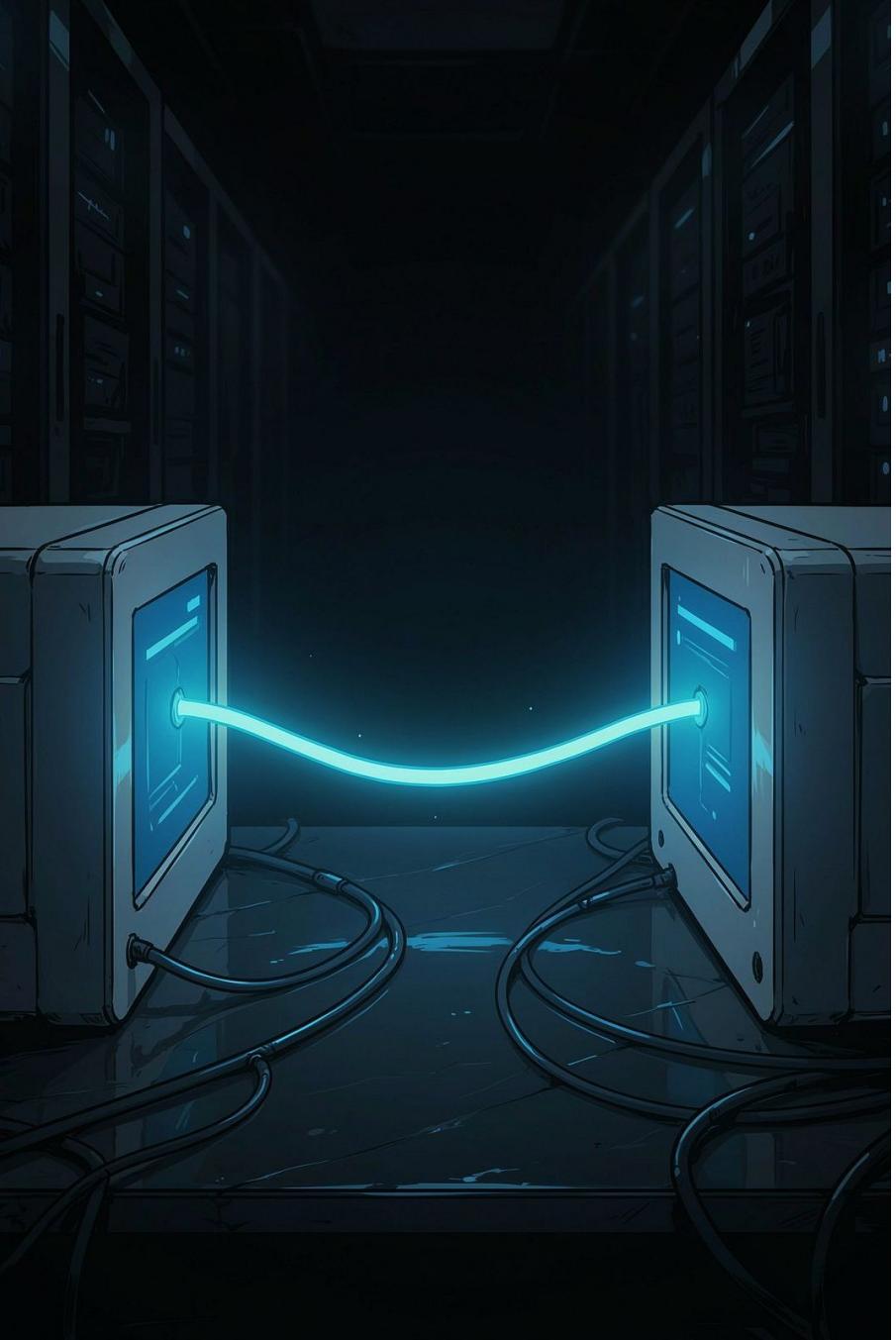
Выполняет вторую команду только если первая завершилась успешно.

Двойная вертикальная черта `||`

Выполняет вторую команду только если первая завершилась с ошибкой.

Backtick `` `cmd` ``

Подстановка вывода команды в строку — аналог `$(cmd)`.



Reverse Shell: что это и зачем

Reverse Shell — интерактивный канал, при котором скомпрометированный сервер сам подключается к машине оператора. В CTF он может стать следующим шагом после подтверждения выполнения кода.

> ### Назначение
>
> Получить интерактивную оболочку с правами веб-процесса для исследования файловой системы контейнера.

> ### Безопасная альтернатива для занятия
>
> Используйте локальные маркеры: вывод `id`, `whoami` или содержимое заранее созданного файла с флагом — без удалённых подключений.

РАЗДЕЛ 4

XML-инъекции

XML применяется в SOAP, SAML, старых интеграциях и офисных документах. Уязвимые парсеры способны читать файлы сервера или обходить аутентификацию.

XXE: внешние XML-сущности

Нормальный XML-профиль

```
<profile>
  <name>student</name>
</profile>
```

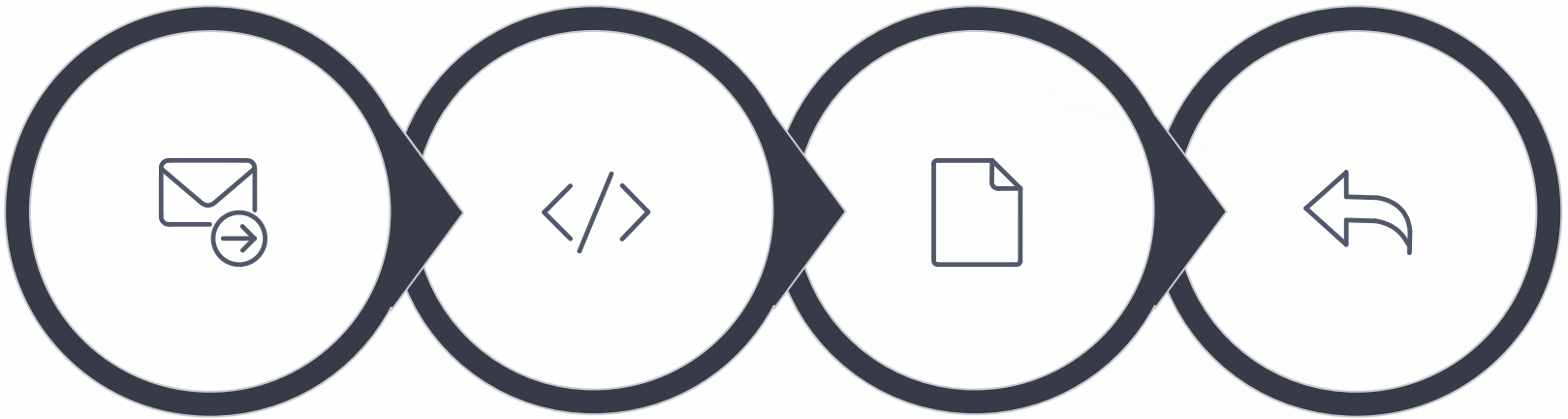
Парсер читает документ и возвращает имя пользователя. Всё безопасно.

XXE-атака

```
<?xml version="1.0"?>
<!DOCTYPE profile [
  <!ENTITY xxe
   SYSTEM "file:///app/flag.txt">
]>
<profile>
  <name>&xxe;</name>
</profile>
```

Если ответ содержит содержимое `/app/flag.txt` — парсер раскрыл внешнюю сущность. В учебном образе флаг кладут в этот файл заранее.

Как работает XXE



Отправка XML

Расширение ENTITY

Чтение flag.txt

Возврат содержимого

Уязвимость возникает, когда XML-парсер настроен на раскрытие внешних сущностей (`SYSTEM` или `PUBLIC`). Безопасная конфигурация — полное отключение DTD или запрет внешних сущностей.

XPath-инъекция

XPath — язык поиска по XML-дереву, аналогичный по назначению SQL. Уязвимый сервер формирует выражение по шаблону:

```
/users/user[login/text() = '<логин>'
      and password/text() = '<пароль>']
```

Нормальный запрос

```
/users/user[
  login/text() = 'admin'
  and password/text() = 'secret'
]
```

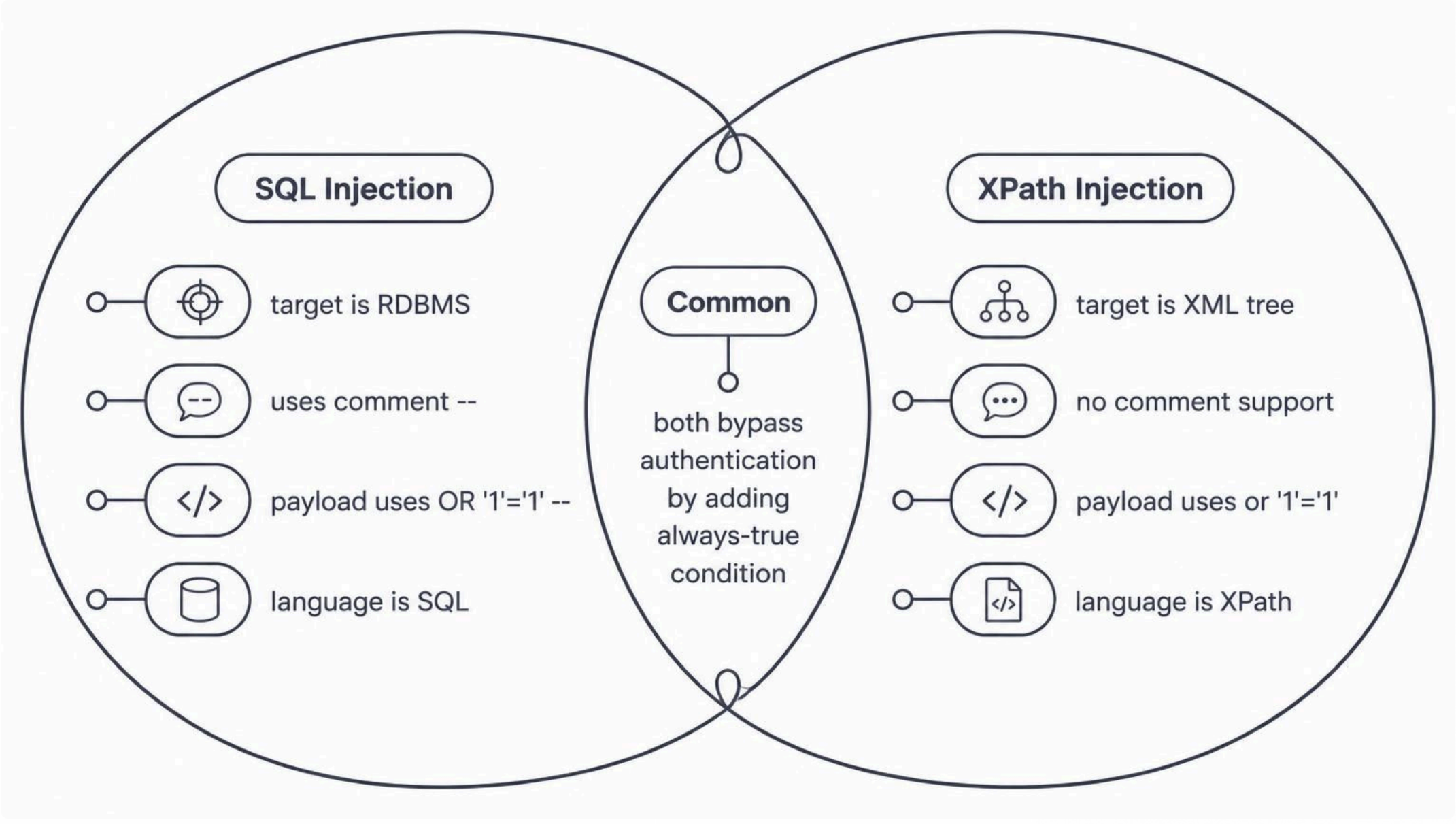
Оба условия должны выполниться — безопасно.

После инъекции ' or '1'='1

```
/users/user[
  login/text() = 'admin'
  and password/text() = ''
  or '1'='1'
]
```

В XPath используют синтаксис XPath: `and`, `or`, `contains()`. SQL-комментарии (`--`) здесь обычно не работают.

SQL vs XPath: сравнение инъекций



РАЗДЕЛ 5

Другие типы инъекций

Принцип инъекции универсален: важно определить, какой интерпретатор разбирает ввод, и подобрать соответствующий синтаксис.



Command Injection

Механизм

Приложение склеивает пользовательский параметр со строкой команды ОС и передаёт её оболочке через `system()`, `exec()` или аналоги. Разделители команд Bash (`;`, `&&`, `||`, `` ` ``) позволяют добавить вторую команду.

Учебная строка

```
127.0.0.1; id
```

Что наблюдать

В ответе появляется вывод второй команды — например, `uid=33(www-data)`. Это подтверждает выполнение произвольной команды с правами веб-процесса.

LDAP Injection

Что такое LDAP

LDAP — протокол доступа к каталогам (Active Directory, OpenLDAP). Приложения используют LDAP-фильтры для поиска пользователей при аутентификации.

Уязвимый фильтр

```
(uid=<ввод>)(password=<ввод>)
```

Инъекционная строка

```
*)(uid=*
```

Результат

Фильтр раскрывается в универсальный поиск, который возвращает всех пользователей каталога. Изменившийся набор результатов — индикатор уязвимости.

NoSQL Injection (MongoDB)

Нормальный запрос

```
db.users.find({
  login: "admin",
  password: "secret"
})
```

MongoDB ищет документ с точным совпадением логина и пароля.

Инъекция через JSON

```
{
  "login": "admin",
  "password": {"$ne": null}
}
```

Оператор `$ne` означает «не равно null» — условие истинно для любого пользователя. Вход выполнен без знания пароля.

Инъекция передаётся в теле JSON-запроса. Ищите API-эндпоинты, принимающие объекты вместо строк.

SSTI: инъекция в серверный шаблон

Server-Side Template Injection возникает, когда пользовательский ввод попадает в шаблонизатор (Jinja2, Twig, Freemarker, Pebble) без экранирования.

Обнаружение

Введите `{{"{{"}}7*7{{"}}"}}` в поле ввода. Если в ответе появляется `49` — шаблонизатор выполнил выражение.

Опасность

Через SSTI возможны: чтение файлов, выполнение команд ОС, RCE — в зависимости от шаблонизатора и его конфигурации.

Типичные движки

Jinja2 (Python), Twig (PHP), Freemarker (Java), Velocity (Java) — у каждого свой синтаксис эксплуатации.



CRLF Injection

Принцип

HTTP-заголовки разделяются последовательностью `\r\n` (CRLF). Если пользовательский ввод попадает в заголовок без фильтрации, атакующий может вставить дополнительные заголовки в ответ сервера.

Учебная строка

```
%0d%0aX-Lab: injected
```

Что наблюдать

В тестовом HTTP-ответе появляется дополнительный заголовок `X-Lab: injected`. Это подтверждает, что ввод интерпретирован как часть HTTP-ответа.

Последствия

HTTP Response Splitting, Cache Poisoning, Cookie Injection — атаки, строящиеся на CRLF.

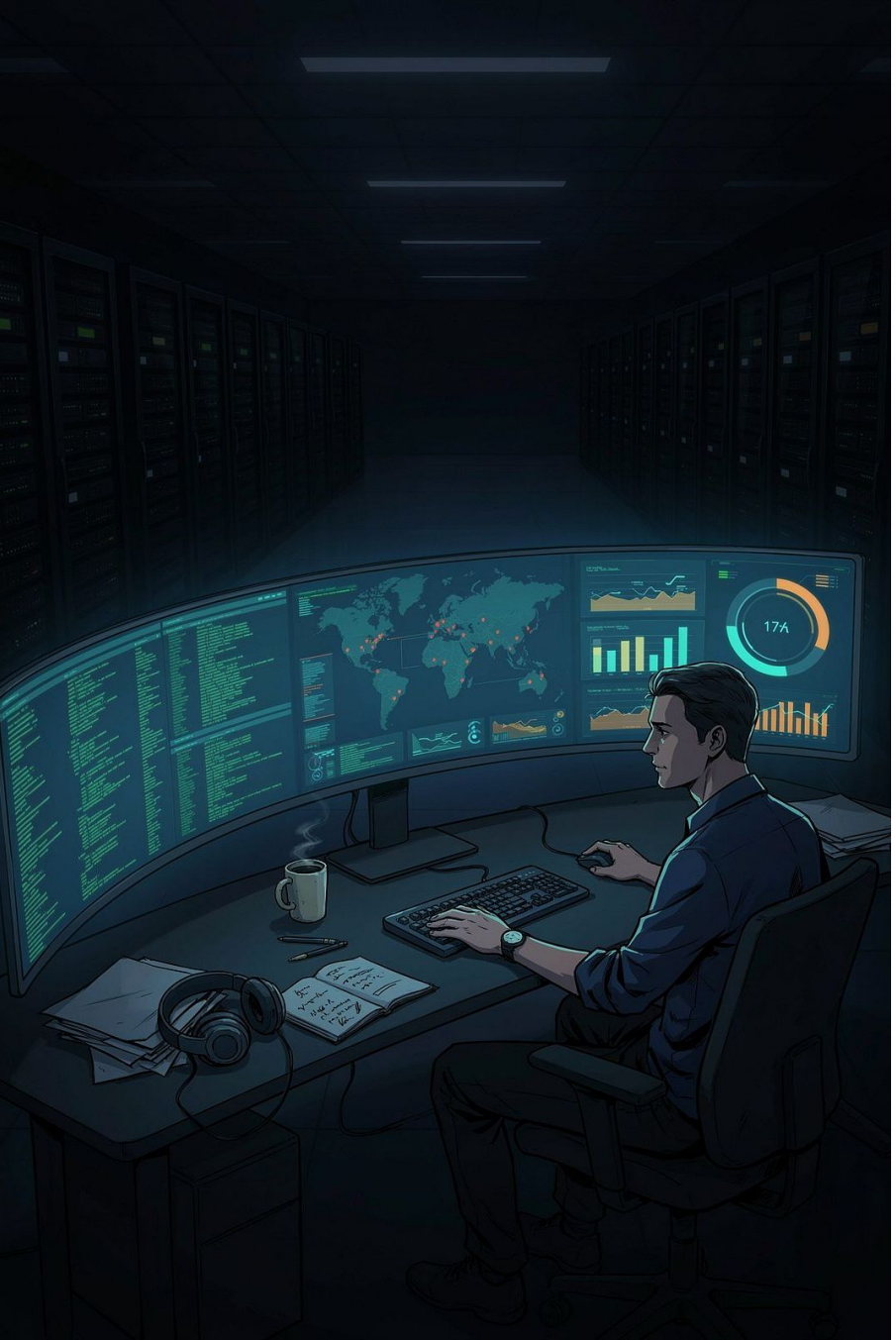
Сводная таблица типов инъекций

Тип	Что склеивает приложение	Учебная строка	Что наблюдать
SQL Injection	Ввод и SQL-запрос	`' OR '1'='1' --`	Вход без пароля, данные другого пользователя
PHP/eval()	Ввод и PHP-код	`1; echo "test"; //`	Произвольная строка в ответе
Command Injection	Параметр и команда ОС	`127.0.0.1; id`	Вывод второй команды
XXE	XML с внешней сущностью	`ENTITY xxe SYSTEM "file://..."`	Содержимое файла в ответе
XPath Injection	Ввод и XPath-выражение	`' or '1'='1`	Вход без пароля
LDAP Injection	Ввод и LDAP-фильтр	`*)(uid=*)`	Расширенный набор результатов
NoSQL Injection	JSON-поле и фильтр MongoDB	`{"$ne": null}`	Вход при неизвестном пароле
SSTI	Ввод и серверный шаблон	`{{"{{"}}7*7{{"}}"}}`	В ответе появляется 49
CRLF Injection	Ввод и HTTP-заголовок	`%0d%0aX-Lab: injected`	Дополнительный заголовок в ответе

Универсальный алгоритм анализа



Этот алгоритм работает для любого типа инъекции. Главное — сначала определить, какой компонент разбирает итоговую строку, и лишь потом подбирать синтаксис.



Зачем всё это важно: статистика уязвимостей

#1

OWASP Top 10

Injection занимает первое место в OWASP Top 10 на протяжении многих лет — самая критичная категория уязвимостей веб-приложений.

94%

Приложений уязвимы

По данным OWASP, 94% протестированных приложений содержат ту или иную форму инъекционной уязвимости.

A03

OWASP 2021

В редакции OWASP Top 10 2021 года инъекции занимают позицию A03:2021.

Защита от инъекций: общие принципы



Параметризованные запросы

Prepared statements разделяют код и данные на уровне драйвера — пользовательский ввод никогда не интерпретируется как часть запроса.



Валидация и санитизация

Проверяйте тип, длину и допустимые символы на стороне сервера. Whitelist (разрешённые значения) надёжнее blacklist.



Минимальные права

БД-пользователь приложения должен иметь только необходимые привилегии: SELECT там, где нужен только SELECT.



Логирование и WAF

Ведите журналы запросов и используйте Web Application Firewall для обнаружения характерных инъекционных паттернов.

Задания для закрепления

ПРАКТИКА

1

SQL: обход и анализ

В SQLInjectionLab войдите в профиль администратора через поле пароля и зафиксируйте итоговый SQL-запрос в полном виде.

2

SQL: модификация запроса

Измените запрос входа так, чтобы он выбирал только `id` и `login`. Объясните, почему обход условия сохраняется.

3

PHP: мини-калькулятор

В отдельном PHP-контейнере соберите калькулятор с `eval()` и подтвердите выполнение строки `1; echo "PHP code executed"; //`

4

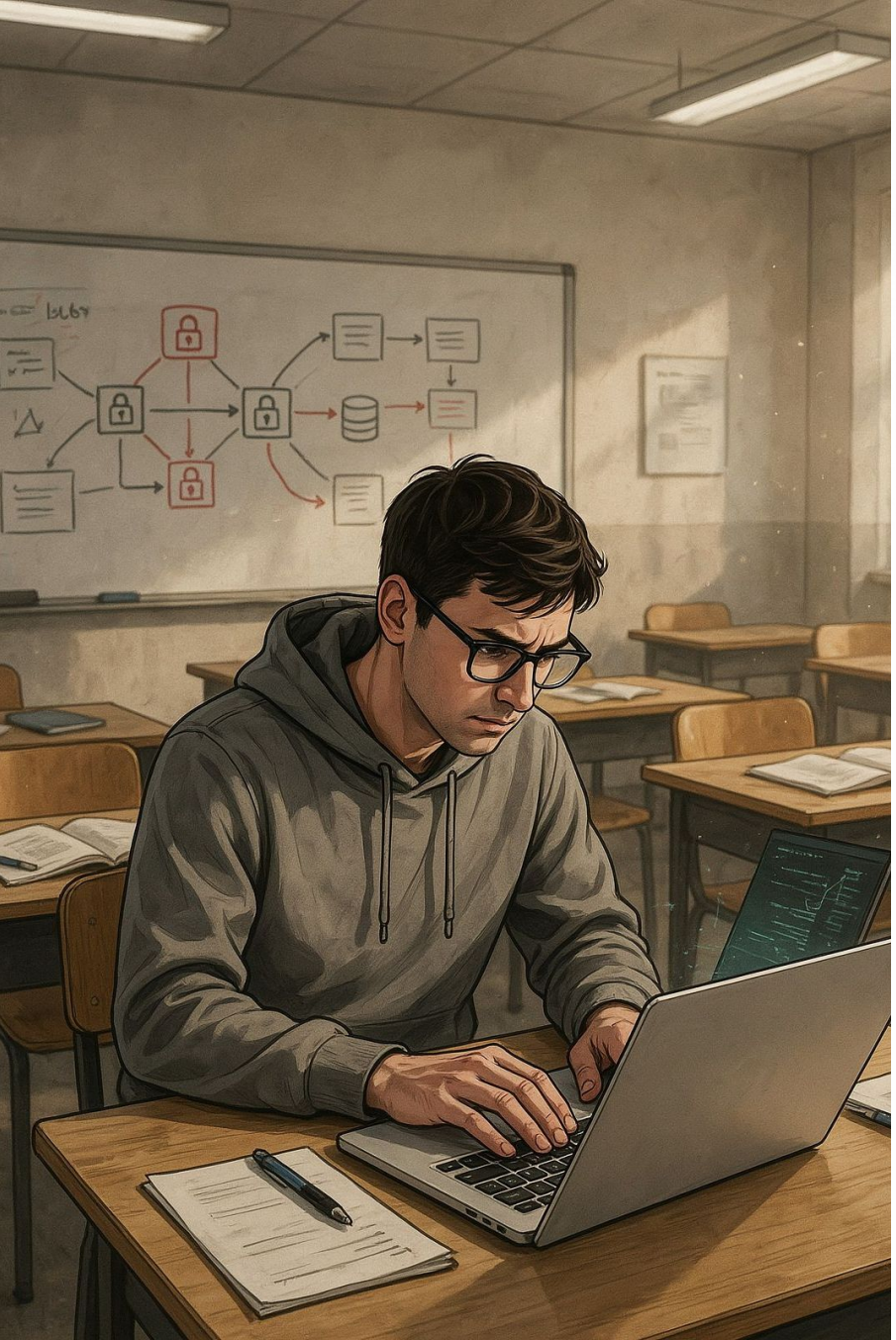
XXE: внешняя сущность

Подготовьте XML с внешней сущностью, ссылающейся на `/app/flag.txt`. Опишите ожидаемый ответ сервера.

5

Таблица инъекций

Для каждой строки из сводной таблицы укажите поле тестового приложения и объясните изменение итогового выражения.



Ключевые выводы лекции

Инъекция – это вопрос контекста

Определите интерпретатор (SQL, PHP, XPath, Bash, LDAP) — и синтаксис атаки станет очевидным.

Ищите точку конкатенации

Любая склейка пользовательского ввода со строкой запроса без параметризации — потенциальная уязвимость.

Практикуйтесь только в изолированной среде

Все эксперименты — только в собственном контейнере. Никогда не применяйте техники вне авторизованных стендов.